

UniVerse

A Campus Companion

Theming & Design System

How one set of tokens controls every color, font and spacing in the app.

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

Flutter (Dart) - Supabase - Firebase FCM - OR-Tools CP-SAT

Owner: Fahmid (shared infrastructure)

Key files: `core/theme/app_colors.dart`, `app_text_styles.dart`, `app_spacing.dart`, `app_theme.dart`

1. The idea (say this first)

UniVerse uses a **centralized design system**. Nothing is hardcoded in screens - every color, text style and spacing value is a **named token** in four files under `core/theme/`. Change one token and the whole app updates. This is a hard project rule: no raw hex, no raw `TextStyle()`, no magic numbers in widgets.

file	responsibility
<code>app_colors.dart</code>	Every color as a named constant (AppColors.primary, bgCard...).
<code>app_text_styles.dart</code>	Every TextStyle (h1, body, badge, countdown...).
<code>app_spacing.dart</code>	Spacing, radii, icon sizes, component heights.
<code>app_theme.dart</code>	Wires all tokens into Flutter's ThemeData (one line in main).

Say this: If the advisor says 'make the orange blue', I change ONE line - AppColors.primary - and all 60+ screens update. No find-and-replace across files.

2. Colors (app_colors.dart)

`abstract class AppColors` holds only `static const Color` values, grouped by purpose so the palette reads like a system, not a random list:

- **Background layers** - bgPrimary (screen) < bgCard (cards) < bgElevated (inputs/chips). A depth stack.
- **Brand** - primary `#FF7A00` (the one personality color), plus primaryDark / primarySoft / primaryMuted for states.
- **Text hierarchy** - exactly four shades: textPrimary, textSecondary, textMuted, textDisabled. Never any other grey.
- **Semantic/status** - success/info/warning/error/done, each with a FULL color (text/icon) and a SOFT color (badge background).
- **Notification type colors** - the 3px left border that tells you the alert type at a glance (cancel=red, room=blue, exam=orange...).
- **Role colors** - student=green, teacher=blue, admin=orange.

```
Container(color: AppColors.bgCard)
Text('LIVE', style: TextStyle(color: AppColors.success))
// opacity helper, no magic numbers:
AppColors.withOpacity(AppColors.primary, 0.15)
```

3. Text styles (app_text_styles.dart)

`abstract class AppTextStyles` defines every `TextStyle` as a named token on a clear scale. Widgets use these instead of writing `TextStyle(fontSize: ...)`.

token group	use
<code>h1 / h2 / h3 / h4</code>	Heading scale - one h1 per screen, h2 for sections.
<code>body / bodyMedium</code>	Reading text + metadata.
<code>chip / label / labelCaps</code>	Small purposeful text (pills, form labels, status badges).
<code>countdown / statNumber</code>	Big display numbers (live-class timer, dashboard stats).
<code>button / link / danger</code>	Interactive text.

You override with `.copyWith()` instead of making a new style: `AppTextStyles.bodySm.copyWith(color: AppColors.error)`. Note: the font family is NOT set here - it is applied globally in app_theme via

GoogleFonts, so every style inherits **Inter** automatically.

4. Spacing (app_spacing.dart)

All paddings, gaps, corner radii, icon sizes and component heights are tokens on a consistent scale (xs4, sm8, md12, lg16, xl20, xxl24...). Radius objects (radiusSm..radiusXxl) + their double values, plus fixed component sizes like buttonHeight, inputHeight, appBarHeight, icon sizes. This keeps every screen on the same rhythm.

5. app_theme.dart - wiring it into Flutter

This is the single entry point that maps all tokens into Flutter's **ThemeData**, so built-in widgets look correct with zero extra styling. Used in main.dart as ``theme: AppTheme.dark``.

What AppTheme.dark configures

- **ColorScheme.dark** - maps semantic roles (primary, surface, error...) to our tokens, so ``colorScheme.primary`` is our orange.
- **textTheme: GoogleFonts.interTextTheme(...)** - sets Inter globally and maps Flutter's named styles (headlineLarge, bodyMedium...) to AppTextStyles.
- **Component themes** - AppBar, BottomNavigationBar/NavigationBar, ElevatedButton (orange), inputs (orange focus border, red error border), chips, progress, checkbox, switch, snackbar, bottom sheet, list tile, popup menu, FAB.

``AppTheme.setSystemUI()`` (called in main) makes the status bar transparent with light icons and locks portrait orientation.

Theme vs custom widgets

ThemeData styles **built-in** Flutter widgets as a baseline. Our custom ``U*`` widgets (UButton, UTextField, UCard) read AppColors/AppTextStyles **directly** for precise control. Both pull from the same tokens, so they always match.

6. Likely defense questions

question	answer
Why tokens, not inline styles?	One source of truth -> consistent UI + one-line global changes. No hunting hex across files.
How is the font set everywhere?	GoogleFonts.interTextTheme() in app_theme - every style inherits Inter; we never set fontFamily per widget.
Light mode later?	Add AppColors light variant + AppTheme.light following the same structure, toggle in main. Structure already supports it.
Why abstract classes?	They are pure namespaces of static consts - you never instantiate AppColors(), you read AppColors.primary.
Icons?	phosphor_flutter, always PhosphorIconsRegular.* - one icon family for consistency.